

Migrating from GBSwagger to SwagDoc

This guide migrates an API written with [Horse](#) from [GBSwagger](#) to SwagDoc, which writes the documentation as an OpenAPI 3 document instead of a Swagger 2.0 one.

What changes

	GBSwagger	SwagDoc for Horse
Specification	Swagger 2.0	OpenAPI 3 or Swagger 2.0, by the <code>SpecVersion</code> property
Documentation	Attributes on the controllers and on the models	Objects of the document, written in code
Routes	Registered by the attributes, in the path registry	Registered in Horse by the application
User interface	Swagger UI	Swagger UI or Scalar
Files of the interface	Loaded from an external address	Loaded from an external address or embedded

The most important difference is the third one. In GBSwagger the `SwagGET`, `SwagPOST` and `SwagPath` attributes **also register the routes** in Horse, so removing them removes the routes of the API. The migration therefore has two parts: registering the routes and documenting them.

1. Dependencies

In the `boss.json` file, replace the dependency:

```
- "github.com/gabrielbaltazar/gbswagger": "^3.0.7",  
+ "github.com/marcelojaloto/SwagDoc": "^2.0.0",
```

In the search path of the project, replace the three folders of GBSwagger by the two of SwagDoc:

```
..\..\modules\gbswagger\Source\Horse;..\..\modules\gbswagger\Source\Core;..\..\modules\gbswagger\Source\Validator;  
+..\..\modules\SwagDoc\Source;..\..\modules\SwagDoc\Integrations\Horse\Source;
```

2. The middleware

```
- Horse.GBSwagger,  
+ Horse.SwagDoc,
```

```

- THorse.Use(HorseSwagger('api/help'));
+ SwaggerConfig.UserInterfaceRoute := '/api/help';
+ SwaggerConfig.DocumentRoute := '/api/help/openapi.json';
+ SwaggerConfig.DiscoverRoutes := False;
+ THorse.Use(HorseSwaggerDoc);

```

`DiscoverRoutes` writes the routes registered in Horse that are not documented yet. It is useful while the migration is not finished, to see what is still missing, and it is turned off when every operation is documented. Keep in mind that the discovery compares the path of the route with the path of the document: when the document declares a server with a base path, like `http://localhost:9000/api`, the paths of the document do not repeat the `/api` prefix and every route looks undocumented to the discovery.

3. The routes

The attributes of GBSwagger register the routes, so the application registers them now. A controller whose methods answer the routes keeps working with a class method that creates it, calls the method and destroys it:

```

class procedure TController.Execute(Request: THorseRequest; Response: THorseResponse;
  const Action: TControllerAction);
var
  Controller: TController;
begin
  Controller := Self.Create(Request, Response);
  try
    Action(Controller);
  finally
    Controller.Free;
  end;
end;

```

The constructor of the controller must be `virtual`, so that `Self.Create` creates the class of the route. With it, each route is registered like this:

```

THorse.Get('/api/tasks',
  procedure(Request: THorseRequest; Response: THorseResponse)
  begin
    TTaskController.Execute(Request, Response,
      procedure(Controller: TController)
      begin
        TTaskController(Controller).List;
      end);
  end);

```

The `initialization` section that registered the controller in GBSwagger is removed:

```
-initialization
- THorseGBSwaggerRegister.RegisterPath(TTaskController);
```

4. The documentation of the operations

Each attribute becomes a property of the objects of SwagDoc. The `Route` method translates a route written with the syntax of Horse, and adds a parameter for each variable of the route.

GBSwagger	SwagDoc
[SwagPath('tasks', 'Tasks')]	SwagDocApi.Route('/tasks') and Operation.Tags.Add('Tasks')
[SwagGET('List of all tasks')]	Path.AddOperation(ohvGet) and Operation.Summary
[SwagPOST('Create a new task')]	Path.AddOperation(ohvPost)
[SwagPUT('{id}', 'Change data')]	SwagDocApi.Route('/tasks/{id}').AddOperation(ohvPut)
[SwagPATCH(...)] , [SwagDELETE(...)]	ohvPatch , ohvDelete
[SwagPathParam('id', 'Task Id')]	Written by <code>Route</code> ; the description is set in <code>Path.Parameters[0]</code>
[SwagParamQuery('count', 'Description', False, False)]	TSwagRequestParameter with <code>InLocation := rpiQuery</code>
[SwagParamBody('Task data', TTaskModel)]	Operation.RequestBody.AddMediaType('application/json').Schema.Name := 'task'
[SwagResponse(200, TTaskModel, 'Task list')]	TSwagResponse with <code>StatusCode</code> , <code>Description</code> and the media type
[SwagResponse(404)]	TSwagResponse with the description of the status

An operation documented with SwagDoc:

```

vOperation := SwaggerDoc.Api.Route('/tasks/{id}').AddOperation(ohvGet);
vOperation.OperationId := 'getTask';
vOperation.Summary := 'Get data for a specific task';
vOperation.Tags.Add('Tasks');

vResponse := TSwaggerResponse.Create;
vResponse.StatusCode := '200';
vResponse.Description := 'Task data';
vResponse.AddMediaType('application/json').Schema.Name := 'task';
vOperation.Responses.Add(vResponse.StatusCode, vResponse);

```

Operations written in code are longer than attributes, and in exchange they reach the whole model of the specification: request bodies with several media types, examples, links, callbacks, webhooks and the reusable objects of the components.

5. The schemas

The attributes of the models are replaced by schemas written with `TJsonSchema` and added to the document as definitions. SwaggerDoc writes them under `definitions` in a Swagger 2.0 document and under `components/schemas` in an OpenAPI 3 one.

GBSwagger	SwaggerDoc
[SwaggerProp('title', 'Description', True, False)]	AddField<string>('title', 'Description') with Required
[SwaggerString(100)]	TJsonFieldString with MaxLength
[SwaggerString(36, 36)]	TJsonFieldString with MinLength and MaxLength
[SwaggerNumber(0, 4)]	TJsonFieldInteger with MinValue and MaxValue
A property of a class type	AddField<T> of the type, or a \$ref to another schema

```

vSchema := TJsonSchema.Create;
vSchema.Root.Description := 'A task of the manager';

vTitle := TJsonFieldString(vSchema.AddField<string>('title', 'Task title description.'));
vTitle.Required := True;
vTitle.MaxLength := 100;

vDefinition := TSwaggerDefinition.Create;
vDefinition.Name := 'task';
vDefinition.JsonSchema := vSchema.ToJson;
SwaggerDoc.Api.Definitions.Add(vDefinition);

```

A schema written by hand references another one with `#/components/schemas/<name>` in an OpenAPI 3 document and with `#/definitions/<name>` in a Swagger 2.0 one. Applications that publish both versions choose the prefix by the `SpecVersion` property of the document.

6. The security

```
- Swagger
- .AddBearerSecurity
- .AddCallback(HorseJWT(TJWTSettings.SECRET_KEY))
```

The middleware of the token is added by the application, to the routes it protects:

```
THorse.Use('/api/tasks', HorseJWT(TJWTSettings.SECRET_KEY));
```

And the document declares the scheme and the operations that require it:

```
vBearer := TSwagSecurityDefinitionHttp.Create;
vBearer.SchemeName := 'bearerAuth';
vBearer.Scheme := 'bearer';
vBearer.BearerFormat := 'JWT';
SwagDocApi.SecurityDefinitions.Add(vBearer);

vRequirement := vOperation.AddSecurityRequirement;
vRequirement.AddScheme('bearerAuth', []);
```

7. The information of the API

```
- Swagger
- .Info
- .Title('Tasks API')
- .Description('The API aims to manage information about tasks.')
- .Contact
- .Name('Marcelo Jaloto')
- .&End
- .&End
- .BasePath('api')
- .AddProtocol(TGBSwaggerProtocol.gbHttp)
+ SwagDocApi.Info.Title := 'Tasks API';
+ SwagDocApi.Info.Description := 'The API aims to manage information about tasks.';
+ SwagDocApi.Info.Contact.Name := 'Marcelo Jaloto';
+ SwagDocApi.AddServer('http://localhost:9000/api', 'Local server');
```

In OpenAPI 3 the host, the base path and the protocol are replaced by the servers. An application that keeps publishing a Swagger 2.0 document uses the `Host`, `BasePath` and `Schemes` properties instead, and `SwaggerDoc` translates them to a server when the document is written as OpenAPI 3.

8. The specification version

The document published by the middleware is written as OpenAPI 3. An API that needs to keep the Swagger 2.0 document during the migration changes one property:

```
SwaggerDocApi.SpecVersion := svSwagger2;  
SwaggerDocConfig.DocumentRoute := '/api/help/swagger.json';
```

The same objects produce both documents, so the two versions can be compared before the old one is dropped.

Checklist

- ☐ The dependency and the search path point to `SwaggerDoc`.
- ☐ Every route that the attributes registered is registered in `Horse`.
- ☐ Every operation is documented, and `DiscoverRoutes` shows nothing left.
- ☐ The paths of the document match the routes, taking the base path of the server into account.
- ☐ The models no longer use the attributes of `GBSwagger`.
- ☐ The middleware of the token protects the routes that required the security.
- ☐ The document is valid in both versions, if the API publishes the two.

A complete example

The [tasks-manager-horse](#) sample was migrated with these steps: seven operations, five schemas, authentication with JWT and an OpenAPI 3.2.1 document.